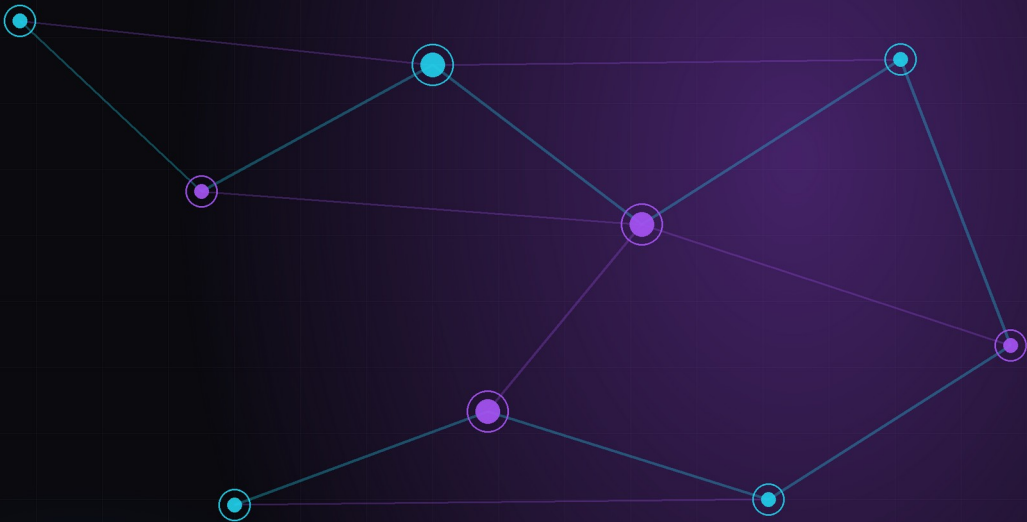


AMD HACKATHON PROJECT



StagePoster

AI-native Music Event Poster Production Engine

From conversational creative planning to candidate image generation, deterministic layout, and multimodal visual review.

- Qwen3.5-9B
- Z-Image-Turbo
- Go Composer
- ROCm / W7900

Project Profile / System Architecture / Model Pipeline / AMD Radeon GPU and ROCm Adaptation

2026.07

00 Executive Summary

StagePoster is an AI-native poster production engine for music events. It turns natural-language requirements into a structured event brief, uses Qwen3.5-9B to generate three confirmable design plans, invokes ComfyUI and Z-Image-Turbo to create text-free candidate key visuals, then uses a deterministic Go Composer to place the real title, date, venue, pricing, logos, and sponsor information. Qwen multimodal review closes the loop with visual quality assessment.

Core innovation

StagePoster is not a simple "write a prompt -> get one image" wrapper. It builds a confirmable, reviewable, and reproducible poster production loop.

Dimension	StagePoster approach
Input method	Conversational requirement collection with missingFields and availableActions
Creative decision	Generate three structured Design Plans before image generation; user confirms one
Image generation	Z-Image-Turbo produces only the Key Visual; text and fake logos are forbidden by default
Layout and branding	Go Composer deterministically places real event text, logos, and sponsors
Quality control	Qwen multimodal review, with up to two RECOMPOSE / REGENERATE rounds
Hardware runtime	AMD Radeon PRO W7900 48 GB with ROCm single-GPU VRAM staging

Contents

- 01 Project Background
- 02 Target Users and Application Scenarios
- 03 System Architecture
- 04 Models and Algorithms
- 05 AMD Radeon GPU / ROCm Adaptation
- 06 Project Status and Technical Highlights

Basis: This document is derived from the StagePoster handoff, official README, and dependency installation guide. Performance figures are only reported when already observed in project materials; unfinished benchmarks are not inferred.

01 Project Background

1.1 Industry problem

Music event posters must combine creative visuals, information hierarchy, and brand assets. Traditional AI image tools can quickly generate visual material, but when asked to produce a complete poster directly, they often create gibberish text, wrong dates, distorted logos, unstable layout hierarchy, and little control before generation.

Pain point	Limit of direct image generation	StagePoster response
Prompt barrier	Users must know style, composition, camera language, and negative prompts	Qwen acts as a conversational art director and automatically structures the brief
Uncontrolled direction	Images are generated first; users react after the fact	Three Design Plans are generated first and confirmed by the user
Text and brand errors	Diffusion models often create gibberish text, fake logos, and wrong numbers	The image model only generates the Key Visual; formal information is composed in Go
Unmeasured quality	A successful model call does not mean the poster is usable	Multimodal visual review plus a bounded Goal Loop
Poor reproducibility	One-off prompts and random outputs are hard to audit	State machine, persistence, review evidence, and best-so-far snapshots

1.2 Project goals

- Lower the prompt and design barrier for professional event poster production.
- Separate creative generation from formal layout to keep visual impact while improving information accuracy.
- Keep the user in control at key points: plan confirmation, candidate selection, and final download.
- Build an automatic visual review loop that can detect issues and trigger recomposition or regeneration.
- Complete an end-to-end, recoverable deployment on AMD Radeon GPU and ROCm.

Product north star

The final deliverable is not a raw generated image, but an auditable package consisting of Final Poster, Thumbnail, and Review Evidence.

02 Target Users and Application Scenarios

2.1 Target users

User group	Core need	StagePoster value
Independent musicians / bands	Quickly create visuals for tours, shows, and releases	No professional prompting required; artists retain direction choice
Livehouses / clubs / venues	High-frequency production of themed event materials	Structured briefs and templates improve production efficiency
Festival and campus teams	Many collaborators, information fields, and revisions	Plan confirmation, candidate selection, and deterministic information layout
Event operations and marketing teams	Need brand, sponsor, and ticketing accuracy	Logos and formal information are accurately composed by the Composer
Design studios / creative agencies	Accelerate concept exploration and first drafts	Three plans plus three candidates expands the creative search space

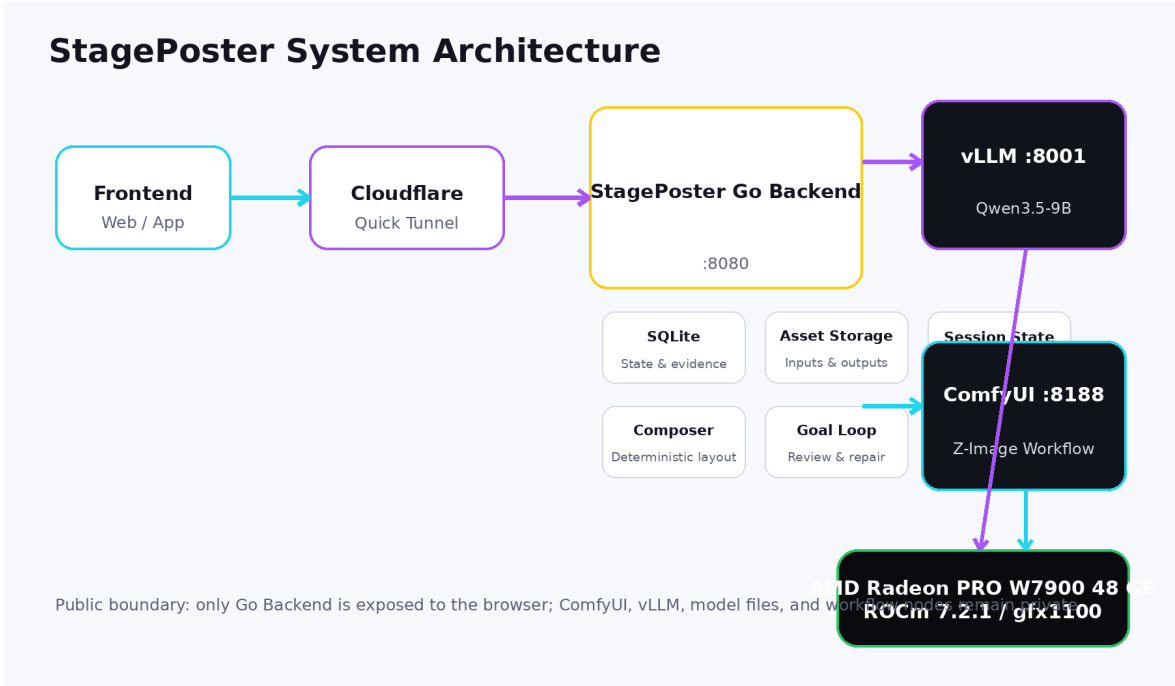
2.2 Typical scenarios

- Concert, livehouse, DJ night, underground party, and tour posters.
- Festival side stages, campus events, co-branded brand activations, and community culture events.
- Rapidly exploring Balanced, Dramatic, Graphic, and other differentiated visual directions for one event.
- Producing formal brand materials when Artist Logo, Event Logo, and Sponsor Logos are available.
- Generating high-quality first-round output in an MVP setting without reference images, using fixed system prompts and structured design rules.

2.3 User journey

Stage	User action	System response
Requirement collection	Describe event, style, genre, and mood	Return missing fields and available actions
Plan decision	Confirm one of three Design Plans	Lock composition, safe zones, palette, and prompt
Visual selection	Review three candidate Key Visuals	Select one Candidate for deterministic layout
Quality closure	Wait for composition and review	Receive final poster, thumbnail, and review status

03 System Architecture



3.1 Core components

Component	Responsibility
Frontend	Collects user input and displays plans, candidates, progress, reviews, and final files.
Cloudflare Quick Tunnel	Provides a temporary HTTPS entry point to the local Go Backend during development.
Go Backend	API, AI Session state machine, persistence, Composer, Goal Loop, Finalize, and GPU coordination.
SQLite / Local Storage	Stores sessions, jobs, candidates, review evidence, assets, and generated poster files.
vLLM + Qwen3.5-9B	Brief parsing, design-plan generation, JSON validation support, and final visual review.
ComfyUI + Z-Image	Runs a versioned workflow to generate three text-free candidate Key Visuals.

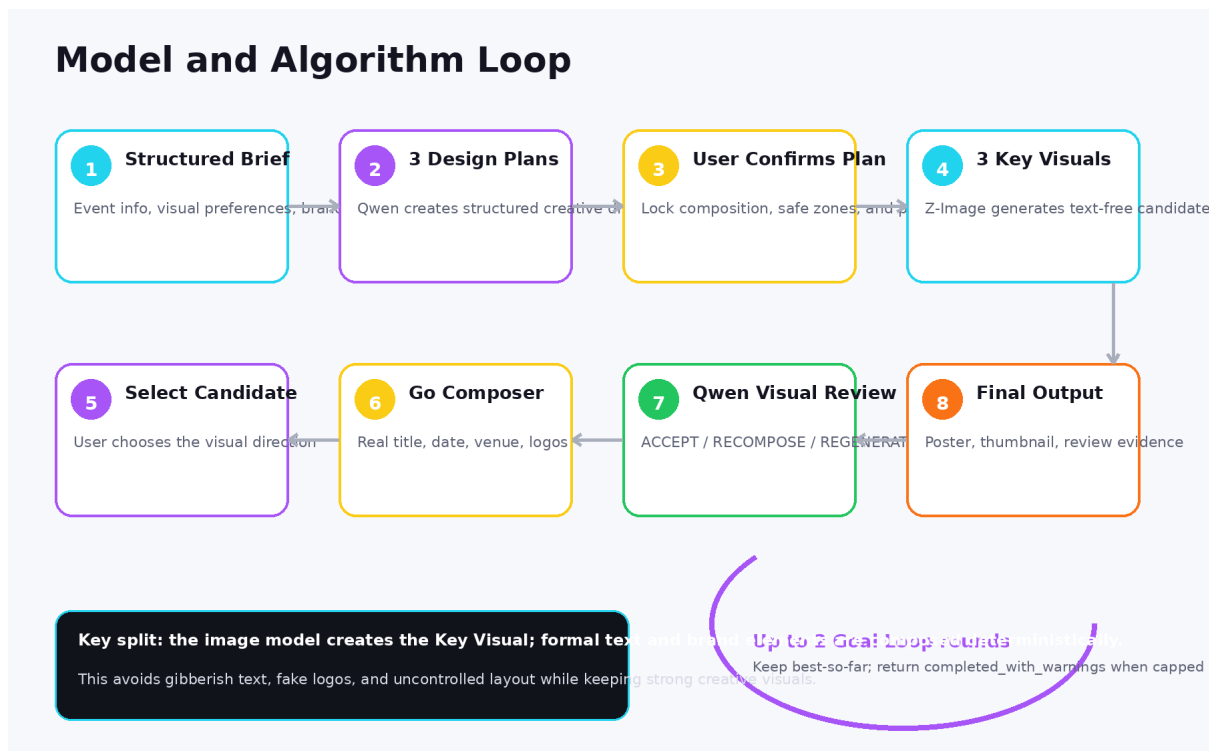
3.2 Security and boundaries

Only the Go Backend is exposed to the browser. ComfyUI, vLLM, SQLite, model files, workflow internals, and node IDs remain private. Development uses Quick Tunnel for temporary public access; production should restrict CORS, enable POSTER_API_TOKEN, and use a fixed Named Tunnel or controlled gateway.

3.3 State machine design

The core state progresses from collecting_brief to awaiting_plan_selection, generating_candidates, awaiting_candidate_selection, looping, and finally succeeded or completed_with_warnings. The frontend must rely on availableActions to determine legal buttons and next steps, rather than guessing process validity.

04 Models and Algorithms



4.1 Qwen3.5-9B: conversational art director and visual reviewer

Qwen3.5-9B is served through vLLM as an OpenAI-compatible API. It handles event brief parsing, missing-field detection, three Design Plans, and the final multimodal visual review. The Design Plan contract includes concept, palette, composition, titleSafeZone, informationSafeZone, positivePrompt, negativePrompt, and composerTemplate.

4.2 Z-Image-Turbo: Key Visual generation

ComfyUI runs the z_image_poster_v1.json API Workflow. The system writes into the bound Positive Prompt node and Seed node to generate three differentiated candidates. The generation stage forbids text, letters, fake logos, watermarks, and gibberish by default, so the model focuses on subject, composition, color, lighting, and atmosphere.

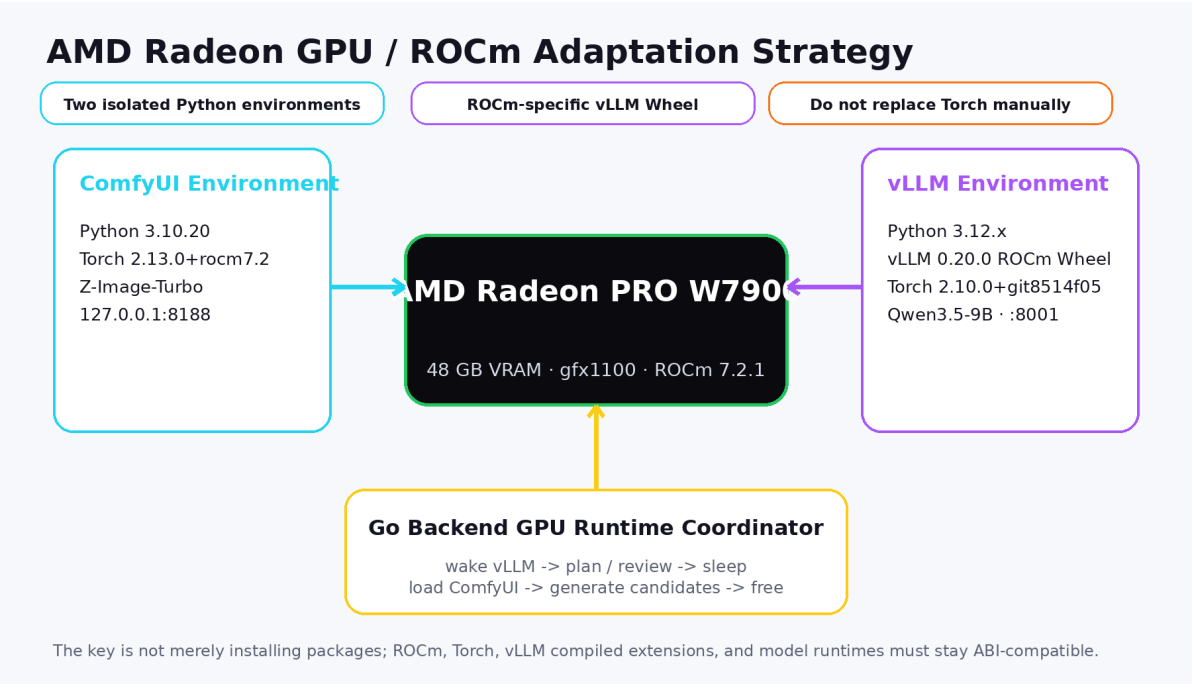
4.3 Go Deterministic Composer: information accuracy layer

After the user selects a Candidate, the Go Composer places the real title, date, time, venue, presale price, door price, Artist Logo, Event Logo, and Sponsor Logos inside safe zones. This deterministic layer addresses diffusion-model unreliability with text, numbers, and brand elements.

4.4 Goal Loop: bounded automatic repair

Qwen returns ACCEPT, RECOMPOSE, or REGENERATE for the composed poster. The system performs up to two repair rounds and saves every snapshot plus the best-so-far result. If the cap is reached but a usable version exists, the system returns completed_with_warnings instead of discarding the best result.

05 AMD Radeon GPU / ROCm Adaptation



5.1 Verified runtime baseline

Item	Verified configuration
Operating system	Ubuntu 24.04.4 LTS / Linux 6.8
GPU	AMD Radeon PRO W7900, 48 GB VRAM
GPU architecture	gfx1100
ROCm / HIP	ROCm 7.2.1 / HIP 7.2.53211
ComfyUI Runtime	Python 3.10.20, Torch 2.13.0+rocm7.2
vLLM Runtime	Python 3.12.x, vLLM 0.20.0 ROCm Wheel, Torch 2.10.0+git8514f05
Go Backend	Go 1.25.0, listening on 127.0.0.1:8080

5.2 Adaptation principles

- ComfyUI and vLLM must use two isolated Python environments to avoid dependency and ABI contamination.
- vLLM must use the ROCm-specific wheel; a default PyPI install can pull CUDA/NVIDIA dependencies or the wrong Torch build.
- Do not separately upgrade or replace Torch after installing vLLM, because that can break the ABI expected by compiled ROCm extensions.
- When the cloud platform already provides a working ROCm image, avoid reinstalling drivers; first verify Python wheel and ROCm compatibility.
- Place models, runtimes, workflows, and critical data under /workspace to reduce recovery cost after instance resets.

05 AMD Radeon GPU / ROCm Adaptation (continued)

5.3 Single-GPU VRAM staging

Qwen3.5-9B and the Z-Image model stack cannot reliably remain resident at the same time on a single 48 GB W7900. The Go Backend treats the GPU as a scheduled shared runtime rather than allowing both model services to occupy VRAM permanently.

Stage	GPU owner	Backend action
Brief collection / Plan generation	vLLM + Qwen3.5-9B	Call wake_up, complete structured reasoning, and persist results
Before candidate generation	Switching window	Call vLLM sleep to release recoverable VRAM
Generate three Candidates	ComfyUI + Z-Image	Load workflow and model stack; execute three differentiated parameter sets
After candidate generation	Release window	Call ComfyUI free to unload model memory
Visual review after Compose	vLLM + Qwen3.5-9B	Wake Qwen again and perform multimodal review
After Goal Loop	Depends on next task	Keep only minimal required services and avoid double residency

Engineering tradeoff

The hackathon MVP prioritizes reliability. If vLLM sleep / wake causes OOM, the system can fall back to forcibly stopping one service and starting the other.

5.4 Stability and recoverability

- Models, workflow, Python runtime, SQLite, and generated artifacts are placed under /workspace to reduce recovery cost after instance destruction.
- Startup checks verify rocm-smi, rocm-info, torch.version.hip, RocmPlatform, and the three local service ports.
- Python locks, Git commit, system versions, and model checksums should be saved to avoid a "runs once but cannot be reproduced" state.
- Quick Tunnel URLs change on every startup and must be extracted dynamically from logs; they should never be hard-coded in frontend source.

5.5 Performance note

Historical samples recorded about 33.6 output tokens/s and 0.088-0.132 s TTFT, but the latest recovered environment switched to vLLM 0.20.0. Therefore the final performance table must be rerun with a reproducible benchmark at submission time. The observed end-to-end Design Plan round trip of about 41 seconds includes orchestration, inference, JSON generation, validation, and persistence, and is not equal to pure decoding throughput.

06 Project Status and Technical Highlights

6.1 Completed and verified

- ComfyUI API Workflow and Go Backend invocation chain.
- SQLite persistence, Asset Registry, Jobs, Candidates, Posters, and Review Evidence.
- Qwen three-plan generation, Plan confirmation, three-candidate generation, and Candidate selection lock.
- Deterministic poster layout, Final Poster, Thumbnail, and multimodal visual review.
- Up to two Goal Loop rounds, RECOMPOSE / REGENERATE / KEEP, best-so-far, and valid warning terminal state.
- Finalize idempotency, concurrency lock, restart recovery, file hash verification, and foreign key checks.
- Real end-to-end run on AMD W7900 + ROCm GPU.

6.2 Latest real E2E reference

Metric	Result
Session	session_8425c32c-a816-4101-b36e-eea19565399c
Plan	gothic-throne-center
Poster	poster_effb132a-6115-4a34-864d-ee65a9fff911
Candidates	3 / 3 ready
Poster Status	succeeded
Finalize Status	completed_with_warnings
Review Rounds	2
Best Score	88

6.3 Technical highlights

Highlight	Description
AI + deterministic systems	The LLM handles creativity and review; Go handles state, data, information accuracy, and idempotency.
Human-in-the-loop	Users make explicit choices at the Plan and Candidate stages.
Evidence-driven output	Results include Review Evidence, round snapshots, and best-so-far.
Native ROCm adaptation	Real inference on W7900 using ROCm PyTorch and ROCm vLLM Wheel.
Recoverable engineering system	Isolated environments, locks, model paths, startup scripts, and health checks support instance recovery.

Current closure focus

Restore and verify the new Cloudflare public URL, complete frontend integration, rerun benchmarks on the current vLLM 0.20.0 environment, and finish the hackathon demo, recording, and submission package.

StagePoster has completed the core backend loop. The next phase is not to redesign the architecture, but to expose the verified capabilities reliably to the frontend and package reproducibility, performance, and demo evidence into a submission-ready artifact.